

Requirement Understanding Document — ParaBank Banking Application

Project: ParaBank Database Testing & SQL Validation

Doc ID	RUD-PARABANK-001
Project	ParaBank Database Testing & SQL Validation
Author	Boopathi T
Role	QA Engineer (Database Testing)
Version	1.1
Date	17-Sep-2026
Status	Reviewed — Locked

Version History

Version	Date	Change	Author
1.0	15-Sep-2026	Initial draft — application understanding, scope, requirements	Boopathi T
1.1	17-Sep-2026	Corrected HSQLDB connection assumption (default port 9001, not 9002) after cross-checking against the official Parasoft repo README	Boopathi T

Why I'm writing this

Before I open a SQL client and start writing queries, I want to actually understand what ParaBank is supposed to do. That's how I'd approach this on a real project too — you don't test what you don't understand.

In a typical project, requirements would normally be provided by a Business Analyst or Product Owner. Since ParaBank is a public demo application and there is no project-specific requirement document available to me, I derived the initial requirements from the application's documented functionality and observed business flows.

These requirements represent my understanding of the application and will be validated against the actual application and database during the testing phases.

What ParaBank actually is

It's Parasoft's demo online banking site — built specifically so QA people have something realistic to practice on. Not a real bank obviously, but the transactions behave like a real bank's would: registration, login, opening accounts, transferring money between your own accounts, paying bills, applying for loans. Under the hood it runs on HyperSQL (HSQLDB), and the app source is open on GitHub, so I can actually look at what's happening instead of guessing blind.

I picked this over something like a static Sakila/Chinook dataset because those don't have an application layer — there's no UI action to trigger and then go verify in the DB. ParaBank does, and that's the whole point of this project: perform an action, then prove the database reflects it correctly.

What I'm covering (and what I'm not)

Testing:

- Registration
- Login
- Viewing account overview
- Opening a new account (Checking/Savings)
- Transferring funds between your own accounts
- Paying a bill
- Requesting a loan
- Finding/searching past transactions
- And for all of the above — checking the database directly, not just trusting what the UI shows

Not testing:

- Performance/load — different project entirely
- Security/penetration testing
- Mobile — there isn't a mobile app
- Visual/UI regression
- Anything multi-branch or multi-currency, since ParaBank doesn't have that

Where this app comes from

Built and maintained by Parasoft. Repo's here: <https://github.com/parasoft/parabank>. I'm just a tester using their public demo instance/local build for practice — this isn't client work, isn't a job. The project focuses on validating application-generated database changes rather than performing isolated SQL exercises.

What I think the business rules are

I'm listing these as what I *believe* the app is supposed to do, based on actually using it and on what Parasoft describes in their docs. If I test something and the app does something different than what's below, that's not automatically a bug — I don't have an official spec to compare against, so I'll note it as an observation and reason about whether it's a defect or just my assumption being wrong.

REQ-01 — Registration: New customer signs up with a unique username; system should create a customer record tied to that login.

REQ-02 — Login: Only someone with a valid, matching username/password should get into account services.

REQ-03 — Open Account: Customer can open a Checking or Savings account. If they fund it from an existing account, that existing account should get debited by the right amount.

REQ-04 — Transfer Funds: Money moves between two accounts that belong to the same customer — source account goes down, destination goes up, by the exact same number. No losing or gaining money in the process.

REQ-05 — Bill Pay: Paying a bill takes money out of the selected account. My assumption (to verify) is that this shouldn't be allowed to push the balance negative.

REQ-06 — Loan Request: Whether a loan gets approved seems tied to comparing the requested amount against what's available in the account — this is one I'm less sure about and want to test carefully.

REQ-07 — Transactions get logged: Every time money moves — transfer, bill pay, loan disbursement — there should be a transaction record created and linked to the right account.

Modules, roughly

Module	What it does
Registration	New user creates a login
Login	Authenticates existing customer
Account Overview	Shows all accounts for the logged-in customer
Open New Account	Creates Checking/Savings, optionally funded from another account
Transfer Funds	Moves money between your own accounts
Bill Pay	Pays a payee from a chosen account
Loan Request	Applies for a loan, approved/denied by some balance logic
Find Transactions	Search past transaction history

Assumptions I'm working with

- I'll have ParaBank running locally (built with Maven, deployed on Tomcat) so I can connect to the DB directly. The official repo says HyperSQL listens on port 9001 by default, so I'll use `jdbc:hsqldb:hsqldb://localhost:9001/parabank`. If 9001 conflicts with something else running on my machine (apparently this happens with the Intel Graphics Command Center service on Windows), I'll reconfigure ParaBank to use 9002 instead, per the repo's port-change instructions — but 9001 is the real default and what I'm assuming unless I hit a conflict.
- All test data will be generated through supported application workflows rather than direct database insertion wherever the application allows it. This keeps the test data representative of actual application transactions.
- No real customer, banking, or financial data will be used. All testing will be performed using the ParaBank demo application and test data.

A couple of constraints worth noting

HSQldb isn't what most companies actually run in production (that'd be more like MySQL, Postgres, SQL Server) — but it's still a real relational database with real SQL, so everything I do here (joins, subqueries, checking constraints) carries over the same way to those platforms. Worth saying that clearly since someone might ask why I didn't just use MySQL from the start — this is what the app comes with.

Also — Parasoft doesn't publish an official schema/DDL file anywhere I could find. So instead of guessing table names, I'm going to actually connect to the running database in the next phase and pull the real schema from `INFORMATION_SCHEMA` . Anything I assumed here about table structure gets checked against that, not the other way around.

How I'll know something's "validated"

- The action in the app either succeeds, or fails the way I'd expect it to (for negative cases)
- I run a SQL query against the DB and it shows what I expected to see
- I write down expected vs. actual, and mark it pass or fail
- If it doesn't match, I log it with the SQL query and result as evidence — not just "it's broken"

Requirement → Test Case tracking (filled in properly during Phase 6)

Req ID	What it covers	Test Case	Status
REQ-01	Registration creates customer record	TBD	Not started
REQ-02	Login only works for valid credentials	TBD	Not started
REQ-03	Open account links to customer, debits funding source	TBD	Not started
REQ-04	Transfer moves exact amount between own accounts	TBD	Not started
REQ-05	Bill pay debits account, balance shouldn't go negative	TBD	Not started
REQ-06	Loan approval tied to balance	TBD	Not started
REQ-07	Every transaction gets logged	TBD	Not started

This table gets filled in with real test case IDs once I get to test design (Phase 6) — right now it's just a placeholder so I remember to come back and link everything up.

Sign-off

Prepared by	Boopathi T (QA Engineer)
Date	17-Sep-2026
Reviewed against	Official ParaBank repo (github.com/parasoft/parabank) and live application behavior
Next phase	Phase 2 — Database Impact Analysis

Next up: Phase 2, working out exactly which tables and fields each of these actions should be touching in the database.